

Google Test Recording

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

4th April 2016

Problem: Large Test Set-Up

- Large Tests are Difficult & Time Consuming to Set-Up
- Yield Curves & Models Must Be Loaded Before Price Testing
- Dependencies are Challenging to Manage

Solution: Test Recording

- A Dependency Loading Feature has been Added to the Library
- A Test Recording & Playback mechanism has also been Added

1 Google Test

- Writing
- Reading
- Recording
- Filtering

2 Loading Dependency Objects

- Yield Curves
- Models

Writing Tests

The general idea is to write a function's parameters to a file ('The Input File') and function results to a file ('The Output File'). We can then read the function parameters from the input file and test that the target function returns the same result as the output file.

Outline

- 1 Include CreateDataFile.h
- 2 Add CreateDataFile namespace
- 3 Write function inputs to file
- 4 Call to the underlying method
- 5 Write function result to file

```
10 #include "tryMirGetParRate.h"
11
12 #include "MFirDateFuncUtility.h"
13
14 #include "CreateDataFile.h"
15 #include "helpers\ValidationHelpers.h"
16
17
18
19 namespace validation_api
20 {
21     using etrading::CreateDataFile;
22
23     /* @brief      validation method for mirGetDate
24      * @param [in]   baseDate      The from date
25      * @param [in]   term          Term added to the from date
26      * @param [in]   slidingRule   Rule that decides if end date should roll forward or backward etc, ie, MF
27      * @param [in]   calendar      Calendar
28      * @param [in]   rollConvention Roll conventions, ie, Normal, IMM, EOM, Lunar, etc
29      * @return       The end date
30      */
31     MBDString tryMirGetDate(const MBDDate& baseDate,
32                             const MBDString& term,
33                             const MBDString& slidingRule,
34                             const MBDString& calendar,
35                             const MBDString& rollConvention)
36     {
37         // Recording of inputs for playback
38
39         if (CreateDataFile::recordEnabled())
40         {
41             CreateDataFile file("tryMirGetDate_inputs");
42             file.write("baseDate", baseDate);
43             file.write("term", term);
44             file.write("slidingRule", slidingRule);
45             file.write("calendar", calendar);
46             file.write("rollConvention", rollConvention);
47         }
48
49         MBDDate mbReturnDate = MFirDateFuncUtility::getDate(baseDate, term, slidingRule, calendar);
50         MBDString ret = mbReturnDate.stringWithFormat("YYYYMMDD");
51
52
53         if (CreateDataFile::recordEnabled())
54         {
55             CreateDataFile file("tryMirGetDate_outputs");
56             file.write("output", ret);
57         }
58
59         return ret;
60     }
61 }
62
```

1

2

3

4

5

Reading Tests

To read a test we need to specify the location of the file containing the function parameters ('The Inputs') and the file containing the function result ('The Output').

Set-Up

- 1 Include ReadDataFile.h & CreateDataFile.h
- 2 Add the ReadDataFile and CreateDataFile namespaces
- 3 Specify the directory location for the test files to be read
- 4 Specify the name of the input file and the output file¹

¹If there are many tests do not specify the index. This is handled separately.

Set-Up for Reading Tests I

```
1 // TestTradeEURAssetSwapSpread.cpp
2
3 #include "CurveOis.h"
4 #include "CurveStd.h"
5 #include "BindFile.h"
6 #include "Dependency.h"
7 #include "CurveTenorBasis.h"
8 #include "MFirCurveFuncUtility.h"
9 #include "MFirCurveGenerateFuncUtility.h"
10 #include "ReadDataFile.h"
11 #include "CreateDataFile.h"
12 #include "MEError.h"
13
14 #include <gTest/gTest.h>
15
16 using google_test::CurveOis;
17 using google_test::CurveStd;
18 using google_test::BindFile;
19 using google_test::Dependency0;
20 using google_test::Dependency1;
21
22 using etrading::ReadDataFile;
23 using etrading::CreateDataFile;
24
25 #define TEST_DIR "Trades\\TestTradeEURAssetSwapSpread\\"
26
27 namespace
28 {
29     // test tolerance
30     const double tolerance = 1e-6; // result is in basis points
31
32     const int minTests = 10;
33
34     //
35     // curve input files
36     //
37     extern const char EURYC_OIS[] = TEST_DIR "EURYC_OIS";
38     extern const char EURYC_1M[] = "";
39     extern const char EURYC_3M[] = "";
40     extern const char EURYC_6M[] = TEST_DIR "EURYC_STD";
41     extern const char EURYC_12M[] = "";
42
43     //
44     // test call input and reference files
45     //
46     extern const char getAssetSwapSpreadInputs[] = TEST_DIR "EURYC_getAssetSwapSpread_inputs";
47     extern const char getAssetSwapSpreadResults[] = TEST_DIR "EURYC_getAssetSwapSpread_results";
48 }
```

Reading Tests

- 1 Load the function arguments from the input file
- 2 Create non-const arguments if required by the calling function
- 3 Read the input file arguments² into the function signature³
- 4 Load the expected result from the result file
- 5 Compare and check that the actual result matches the expected result stored in the output file

²The [] operator has been overloaded for this purpose e.g. line 97 inputFile['curveid']

³If a particular function signature needs extensive testing it would be sensible to write a test fixture class constructor to do this

```
69 TEST_F( TestTradeAssetSwapEUR, CheckAssetSwapSpreadsAgainstSnapshot )
70 {
71     int i = 0;
72     try
73     {
74         for ( i = 0; ; ++i)
75         {
76             // Load the input file 1
77             const ReadDataFile::Load inputFile( MBCreateFile::makeFilename(getAssetSwapSpreadInputs, i) );
78
79             // Input file returns const types, some of which need to be non-const for our function call below
80             double cleanBondPrice = inputFile["cleanBondPrice"];
81             double fixedRate = inputFile["fixedRate"];
82             DateVector fixedAccrualDates = inputFile["fixedAccrualDates"];
83             DateVector fixedPaymentDates = inputFile["fixedPaymentDates"];
84             DateVector floatFixingDates = inputFile["floatFixingDates"];
85             DateVector floatAccrualDates = inputFile["floatAccrualDates"];
86             DateVector floatPaymentDates = inputFile["floatPaymentDates"];
87
88             // Call the asset swap spread function
89             const double assetSwapSpread = MFirCurveFuncUtility::getAssetSwapSpread(
90                 cleanBondPrice, // non-const needed
91                 fixedAccrualDates, // non-const needed
92                 fixedPaymentDates, // non-const needed
93                 floatFixingDates, // non-const needed
94                 floatAccrualDates, // non-const needed
95                 floatPaymentDates, // non-const needed
96                 getRoot(),
97                 inputFile["curveid"],
98                 fixedRate, // non-const needed
99                 inputFile["fixedDaycount"],
100                 inputFile["floatDaycount"],
101                 inputFile["interpolation"],
102                 inputFile["foreCurveName"],
103                 inputFile["dfCurveName"],
104                 inputFile["isFWDInter"],
105                 inputFile["useFirstFixing"],
106                 inputFile["firstFixing"],
107                 inputFile["useLastFixing"],
108                 inputFile["lastFixing"] );
109
110             // Read the results file 4
111             const ReadDataFile::Load resultFile(MBCreateFile::makeFilename(getAssetSwapSpreadResults, i));
112             const double expectedRate = resultFile["result"];
113
114             // Compare
115             EXPECT_NEAR( assetSwapSpread, expectedRate, tolerance );
116         }
117     }
118     catch( const ReadDataFile::LoadError& )
119     {
120         EXPECT_GE(i, minTests);
121     }
122     catch( const MEEError & m )
123     {
124         throw( m.getMsg() );
125     }
126     catch( const std::exception & e )
127     {
128         throw( e.what() );
129     }
130 }
```

Outline

- 1 Call the **mirRecord** function in **Debug mode** to start recording
- 2 Call the function that you wish to record from Excel
- 3 View of generated input- and output files

mirRecord([Enable], [Folder], [Repeat], [StartIndex], [MaxIndex])

- Enable: Enable Test Recording, True or False, defaults to false
- Folder: Record folder, defaults to the **%TEMP%**⁴ location
- Repeat: Enable repeat mode and test indexing, defaults to false
- StartIndex: The start index for repeat mode, defaults to 0
- MaxIndex: The maximum number of tests⁵, defaults to 10

⁴This environment variable is set on all PCs

⁵After the maximum number of tests have been recorded the index will reset to the start index and new tests will begin to overwrite. This is to prevent infinite loops.

EUR Swaps

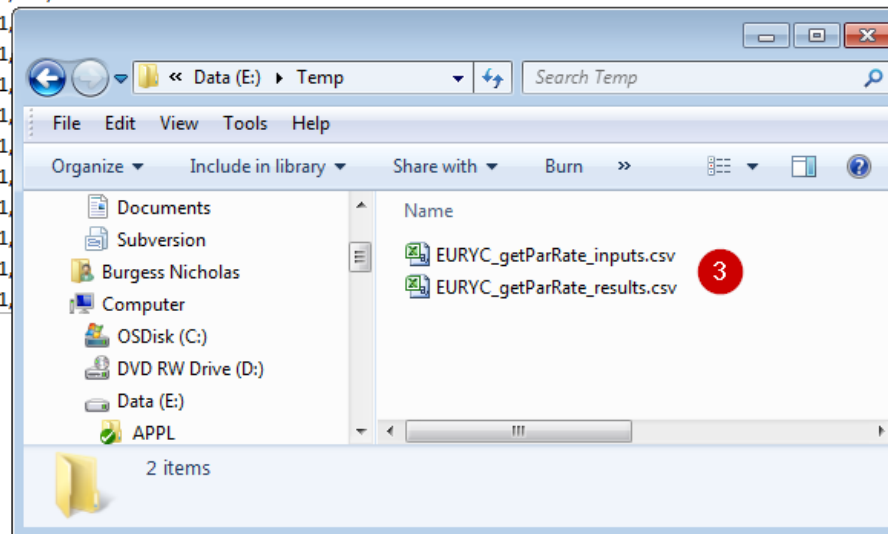
mirRecord("Enable","Folder","Repeat","StartIndex","MaxIndex")

PV

-312,926

Recording Enabled, Output Folder: E:\temp 1

	Effective Date	Effective Lag	Notional	Currency	Pay / Rec Fixed Rate	Maturity Or Tenor	Fixed Rate	Float Index	Float Spread	Float First	CSA	Par Rate	Present Value	PV01
1	Tue 23-Feb-16	2D	1,000,000	EUR	=mirGetParRate4(\$B6,\$G6,\$AM6,\$N6,\$O6,\$P6,\$Q6,\$R6,\$S6,\$T6,\$U6,\$V6,\$Y6,\$Z6,\$AA6,\$AB6,\$AC6,\$AD6,\$AE6,\$AF6,\$AH6,\$AI6,\$AJ6,\$AK6)									
2	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER 2	2Y	1.00000%	6ML	0.00		NOCSA	-0.13300%	-22,791	201
3	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	3Y	1.00000%	6ML	0.00		NOCSA	-0.08700%	-32,910	303
4	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	4Y	1.00000%	6ML	0.00		NOCSA	-0.00800%	-40,682	404
5	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	5Y	1.00000%	6ML	0.00		NOCSA	0.09300%	-45,732	504
6	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	6Y	1.00000%	6ML	0.00		NOCSA	0.20200%	-48,247	605
7	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	7Y	1.00000%	6ML	0.00		NOCSA	0.32100%	-47,815	704
8	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	8Y	1.00000%	6ML	0.00		NOCSA	0.44400%	-44,630	803
9	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	9Y	1.00000%	6ML	0.00		NOCSA	0.56100%	-39,518	900
10	Tue 23-Feb-16	2D	1,000,000	EUR	PAYER	10Y	1.00000%	6ML	0.00		NOCSA	0.66700%	-33,156	996
11	Tue 23-Feb-16	2D	1								DCSA	0.76300%	-25,831	1,090
12	Tue 23-Feb-16	2D	1								DCSA	0.84700%	-18,094	1,183
13	Tue 23-Feb-16	2D	1								DCSA	0.91775%	-10,476	1,274
14	Tue 23-Feb-16	2D	1								DCSA	0.97836%	-2,951	1,364
15	Tue 23-Feb-16	2D	1								DCSA	1.03000%	4,355	1,452
16	Tue 23-Feb-16	2D	1								DCSA	1.06960%	10,704	1,538
17	Tue 23-Feb-16	2D	1								DCSA	1.10207%	16,568	1,623
18	Tue 23-Feb-16	2D	1								DCSA	1.12836%	21,911	1,707
19	Tue 23-Feb-16	2D	1								DCSA	1.14941%	26,740	1,790
20	Tue 23-Feb-16	2D	1								DCSA	1.16600%	31,069	1,872

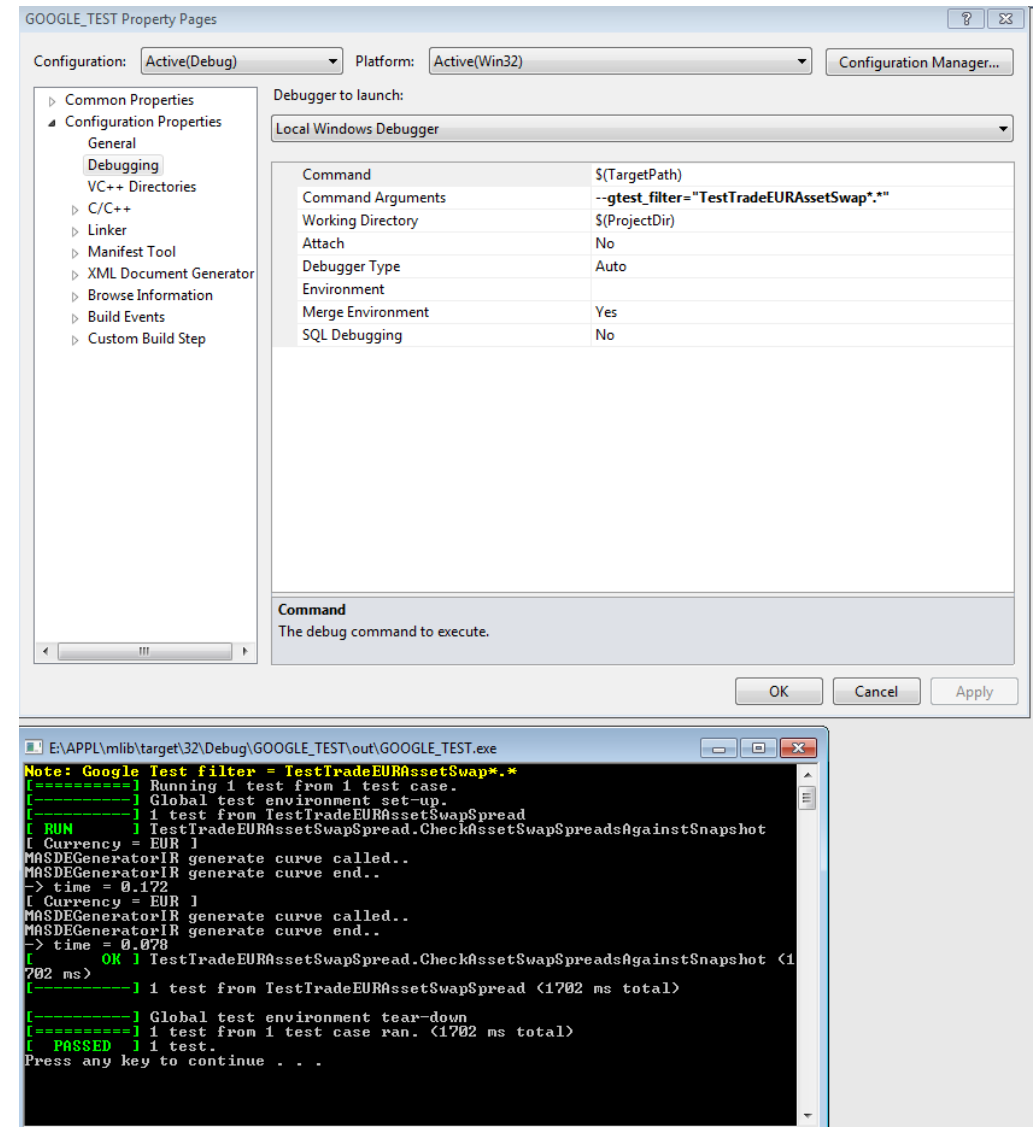


Filtering Tests

- Using `-gtest_filter="*. *_.*"`
- Can filter to include `*.*` or exclude `*.*` tests
- Must name tests systematically

Examples

- 1 Run All Trades
`-gtest_filter="TestTrade*.*"`
- 2 Exclude All Trades
`-gtest_filter="-TestTrade*.*"`
- 3 Run All JPY Trades
`-gtest_filter="TestJPYTrade*.*"`
- 4 Exclude Yield Curve Tests
`-gtest_filter="-TestCurve*.*"`



Automated Yield Curve Loading

- 1 Yield curve constructors are used to load yield curves from input files
- 2 Using Google test fixtures i.e. `TEST_F` test classes inherit these yield curve constructors
- 3 We can nest constructors using inheritance to load all dependency yield curves
- 4 Macros have been written to do this automatically, see `dependency.h` in the Google test project

Example

- 1 List yield curve input file paths
- 2 Call the EUR curve macro
- 3 Quote the Test Fixture name in the Yield Curve macro
- 4 Call the Test Fixture

Note: Macros for other currencies are available

```
31 //  
32 // Curve input files.  
33 //  
34 extern const char DIR_EURYC_OIS[] = TEST_DIR "EURYC_OIS";  
35 extern const char DIR_EURYC_1M[] = "";  
36 extern const char DIR_EURYC_3M[] = "";  
37 extern const char DIR_EURYC_6M[] = "";  
38 extern const char DIR_EURYC_12M[] = "";  
39 //  
40 //  
41 // test call input and reference files  
42 //  
43 extern const char getParRateInputs[] = TEST_DIR "EURYC_getParRate_inputs";  
44 extern const char getParRateOutputs[] = TEST_DIR "EURYC_getParRate_results";  
45 //  
46 extern const char getPvInputs[] = TEST_DIR "EURYC_getSwapPV_inputs";  
47 extern const char getPvOutputs[] = TEST_DIR "EURYC_getSwapPV_results";  
48 }  
49 //  
50 //  
51 namespace google_test  
52 {  
53 //  
54 // Construct dependent curves from curve source files by calling the curve constructor methods  
55 //  
56 // All tenor curve name MUST be defined for the ADD_EUR_CURVE_DEPENDENCIES macro to work.  
57 // If any curve is not in use and not defined, simply assign a "" to the curve name.  
58 //  
59 ADD_EUR_CURVE_DEPENDENCIES TestTradeEUROISParRate DIR_EURYC_OIS, DIR_EURYC_1M, DIR_EURYC_3M, DIR_EURYC_6M, DIR_EURYC_12M  
60 //  
61 //  
62 // Call Test Fixture for OIS Swap par rate  
63 //  
64 //  
65 TEST_F( TestTradeEUROISParRate, OISSwapParRate )
```

Macro: ADD_EUR_CURVE_DEPENDENCIES

The macro will build the test fixture constructor and nest all the yield curve constructors using class inheritance. The inherited yield curve constructors will read and load these yield curves as required.

Automated Model Loading

- ① Using a similar method to the auto loading of yield curves we could also load models
- ② See dependency.h in the GOOGLE_TEST project
- ③ Model class constructors could be set-up to load a model
- ④ Google test fixtures i.e. TEST_F could then inherit the model constructor
- ⑤ Multiple models and yield curves could be loaded using nested inheritance

The End